

CO2 - ASSESSMENT TOOL 01

Analytical Problem Solving in Database Management Systems

Tool Name	CO2 - Assessment Tool 01
Name	M. Yogeshwaran
Reg No	192524363
Course Name	Database Management System
Course Code	CSA0507

1. VIEW — HighSalaryView (Salary Above Department Average)

A VIEW is a virtual table defined by a stored query. It does not hold data itself; every time it is queried, the underlying SELECT is re-executed against the base tables. Here it is used to permanently expose the set of employees earning more than their own department's average salary.

CREATE VIEW Statement

```
CREATE VIEW HighSalaryView AS
SELECT e.EmpID, e.Name, e.Dept, e.Salary
FROM EMPLOYEE e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM EMPLOYEE e2
    WHERE e2.Dept = e.Dept
);
```

The inner subquery is correlated: for every outer row e, it recomputes the average salary of only that employee's department (e2.Dept = e.Dept), then the outer WHERE keeps rows whose salary beats that average.

Using the View

```
-- Retrieve everyone in the view
SELECT * FROM HighSalaryView;

-- Retrieve only high earners in a specific department, ordered
SELECT Name, Dept, Salary
FROM HighSalaryView
WHERE Dept = 'IT'
ORDER BY Salary DESC;
```

2. Relational Algebra Division — Students Enrolled in ALL Semester-1 Courses

Division ($\div$) answers exactly this kind of “for all” question: given a relation $R(A,B)$ and a relation $S(B)$, $R \div S$ returns every value of A that is paired with every value of B in S. Here A = student, B = course, and S is the set of courses offered in Semester 1.

Setup

- ENROLLED(StudentID, CourseID) — which students have taken which courses
- COURSE(CourseID, Semester) — which semester each course belongs to

Relational Algebra Formulation

```
S1_Courses  $\leftarrow \pi$  CourseID (  $\sigma$  Semester = 'Semester 1' (COURSE) )
Enroll_S1  $\leftarrow \pi$  StudentID, CourseID ( ENROLLED  $\bowtie$  S1_Courses )
Result  $\leftarrow$  Enroll_S1  $\div$  S1_Courses
```

Step 1 projects the CourseID of every course offered in Semester 1. Step 2 restricts ENROLLED to only those (StudentID, CourseID) pairs whose course is in that Semester-1 set. Step 3 divides that relation by S1_Courses, leaving only the StudentIDs paired with every single CourseID in S1_Courses — i.e., students who enrolled in ALL Semester-1 courses.

Equivalent SQL (Division via Double NOT EXISTS)

SQL has no direct $\div$ operator, so division is standardly expressed as “no course exists that the student did NOT take”:

```
SELECT DISTINCT e.StudentID
FROM ENROLLED e
WHERE NOT EXISTS (
    SELECT c.CourseID
    FROM COURSE c
    WHERE c.Semester = 'Semester 1'
    AND NOT EXISTS (
        SELECT 1 FROM ENROLLED e2
        WHERE e2.StudentID = e.StudentID
        AND e2.CourseID = c.CourseID
    )
);
```

3. GROUP BY / HAVING — Departments With Average Salary Above 60000

GROUP BY collapses EMPLOYEE rows into one row per department; the aggregate AVG(Salary) is then computed per group. HAVING filters on that aggregate — unlike WHERE, it can reference AVG() because it runs after grouping.

```
SELECT Dept, AVG(Salary) AS AvgSalary
FROM EMPLOYEE
GROUP BY Dept
HAVING AVG(Salary) > 60000
ORDER BY AvgSalary DESC;
```

If only the department names (not the computed average) are needed, project just Dept; the HAVING clause still works the same way since it evaluates the aggregate internally before the final SELECT list is applied.

4. Correlated Subquery — Products With Total Sales Quantity Above 100

A correlated subquery references a column from the outer query, so it is logically re-evaluated once per outer row (here, once per product). This is an alternative to a GROUP BY/HAVING approach and is useful when the filtering condition needs to sit alongside other per-row logic.

```
SELECT p.PID, p.Name, p.Price, p.Category
FROM PRODUCT p
WHERE (
    SELECT SUM(s.Qty)
    FROM SALES s
    WHERE s.PID = p.PID
) > 100;
```

The inner SELECT SUM(s.Qty) is correlated to the outer PRODUCT row through s.PID = p.PID: for each product p, it sums only that product's sales quantities, and the outer WHERE keeps the product if that total exceeds 100. Products with no matching SALES rows return NULL from SUM() and are naturally excluded, since NULL > 100 is unknown, not true.

Equivalent Non-Correlated Version (for comparison)

```
SELECT p.PID, p.Name, p.Price, p.Category
FROM PRODUCT p
```

```

JOIN (
    SELECT PID, SUM(Qty) AS TotalQty
    FROM SALES
    GROUP BY PID
    HAVING SUM(Qty) > 100
) s ON s.PID = p.PID;

```

5. DDL Constraints — LIBRARY(BookID, Title, AuthorID, Price)

Constraint Definitions

- **PRIMARY KEY:** uniquely identifies each row in a table; implies NOT NULL + UNIQUE together. A table has at most one primary key (which may span multiple columns).
- **FOREIGN KEY:** enforces referential integrity by requiring a column's value to match a value in a referenced table's primary/unique key, preventing orphaned references.
- **UNIQUE:** ensures all values in a column (or column combination) are distinct across rows, but unlike PRIMARY KEY it allows a single NULL (per most RDBMS).
- **CHECK:** restricts the allowed values for a column via a boolean condition evaluated on every insert/update, e.g. enforcing a positive price.
- **NOT NULL:** forbids NULL in a column, forcing every row to supply a value for it.

DDL Statements

```

CREATE TABLE AUTHOR (
    AuthorID    INT PRIMARY KEY,
    AuthorName  VARCHAR(100) NOT NULL
);

CREATE TABLE LIBRARY (
    BookID      INT PRIMARY KEY,
    Title       VARCHAR(150) NOT NULL,
    AuthorID    INT,
    Price       DECIMAL(8,2) NOT NULL CHECK (Price > 0),
    CONSTRAINT uq_title UNIQUE (Title),
    CONSTRAINT fk_author FOREIGN KEY (AuthorID)
        REFERENCES AUTHOR(AuthorID)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

```

BookID is the PRIMARY KEY, guaranteeing uniqueness and non-null identification of every book. Title is NOT NULL and additionally UNIQUE, so no two books can share a title and every book must have one. AuthorID is a FOREIGN KEY referencing AUTHOR(AuthorID); ON DELETE SET NULL means deleting an author un-links their books rather than deleting them, and ON UPDATE CASCADE keeps AuthorID in sync if an author's ID changes. Price is NOT NULL and constrained by CHECK (Price > 0) to reject zero or negative prices.